

Introduction to Rust & eBPF with Rust

Pulumati Ram

2026-05-11

Realtek Semiconductor Corporation

> Disclaimer <

Focus on systems evolution, not a language war

- Rust in the Linux kernel -
- Direction and practical utility -

Part #1

Rust as a systems programming language

zero-cost abstraction, Borrow checker, memory layout ctrl, Fearless concurrency

Part #2

Rust in the Linux kernel

chronological update, build, adoption map

Part #3

eBPF programming with Rust

Aya framework, observability case study

1. Why Rust : A Systems Programmer's Perspective:

Memory Safety • Compiler Guarantees • Modern Concepts •
performance

Part 1: Rust as Systems Programming Language

The Problem: Why Another Systems Language?

- `C` is the de-facto industry standard for systems programming.
- `C` built systems (can contain errors that are easy to make and difficult to detect even with rigorous code review)
- Safety of programs is developer-dependent (→ adds complexity and cost of maintenance)
- Linux kernel: evolved over 30+ years, there is a growing concern that new developers are less interested in working with the risks associated with “manual” C.

Rust

- Language philosophy aims to address the above `C` challenges, with out compromising on security, performance or reliability.
- Linux Kernel adoption: Purpose make OS more stable, secure and maintainable. **Not intended for replacing `C`** Intended as a peer language, mainly for :
 - Device Drivers
 - File Systems
 - New sub-systems.
- Kernel maintainers spend a big part of their time reviewing code for memory leaks or pointer errors. Rust adoption would significantly reduce time in fixing basic memory bugs.
- Survey that pushed for Rust's adoption as second language into Linux Kernel:

1.1 The eternal memory bug

The numbers haven't moved in 20 years

- **70 %** of Microsoft CVEs are memory safety bugs
[Microsoft Security Response Centre, 2019](#)
- **67 %** of Linux kernel CVEs are memory safety violations
[Gaynor & Thomas, Linux Security Summit, 2019](#)
- Chrome: **70 %** of high-severity bugs are memory safety issues
[Google Project Zero, 2020](#)
- Android: memory unsafety caused **\$68B** in security costs est. 2019
[Android team blog, 2021](#)

Better tooling (ASAN, Coverity, sparse) **reduces** the rate — it does not **eliminate** the class. The only way to eliminate a class of bugs is to make them unrepresentable in the type system.

The root causes

Cause	C has no protection against...
Use-after-free	accessing freed memory via a dangling pointer
Buffer overflow	writing past the end of an allocation
Data race	two threads accessing shared memory without synchronisation
Null deref	dereferencing a pointer that might be <code>NULL</code>
Uninit. read	reading before a write path has initialised a field
Integer overflow	signed overflow, UB, silent wrong results

1.1 The eternal memory bug

Rust **eliminates every row in this table** at compile time, with zero runtime overhead.

Cause	C has no protection against...
Use-after-free	accessing freed memory via a dangling pointer
Buffer overflow	writing past the end of an allocation
Data race	two threads accessing shared memory without synchronisation
Null deref	dereferencing a pointer that might be <code>NULL</code>
Uninit. read	reading before a write path has initialised a field
Integer overflow	signed overflow, UB, silent wrong results

1.2 Space before Rust

Safe but slow (GC languages)

- Java, Go, Python : garbage collector guarantees no help for use after free (UAFs).
- GC introduces pauses in program execution => unpredictable latency, large run-times.
- These languages unsuitable for kernel code, real-time, interrupt handlers

Fast but unsafe (C, C++) (depends on developer)

- Maximum control, minimum overhead
- **Cost**: all memory bugs are the programmer's problem
- 30+ years of CVEs are the empirical evidence

Rust closes the gap: **safety without a GC**, performance matching C, zero-cost abstractions.

Jung et al., POPL 2018 — “the holy grail of PL research”

	Memory safe	No GC / no runtime
C / C++	✗	✓
Rust	✓	✓
Java / Go	✓	✗

Rust uniquely occupies the upper-right cell.

Rust is not just another general purpose language, it represents a new point in systems programming design space. With Safety and Performance.

2. Ownership : Memory Safety at Compile Time

2.1 The three ownership rules

Rust's ownership system it's a type theoretic answer to manual memory management.

- It's a memory management model where each value has a single owner at a time, and the Compiler enforces rules about ownership/borrowing/lifetimes.
- Shifts memory safety from a runtime concern (like GC) to a compile time verification handled by the type system.

Rule 1 : Each value has exactly one owner

Rule 2 : There can only be one owner at a time (ownership can be moved).

```
let s1 = String::from("hello"); // s1 owns the heap data
let s2 = s1; // ownership MOVES to s2
// println!("{}", s1); // ← compile error: s1 was moved
```

The compiler tracks ownership **statically**. No runtime bookkeeping.

Rule 3 : When the owner goes out of scope, the value is dropped

```
{
    let buf = alloc_dma_buf(); // allocation
} // ← Drop::drop() called HERE, automatically
// No free() to forget. No leak possible.
```

-> Shifts memory safety from a runtime concern (like garbage collection) to a compile-time verification problem handled by the type system.

2.2 Ownership vs C: use-after-free

Most common kernel CVE class : caught at compile time in Rust.

- This is not a bug detection problem.
- This is program expressiveness problem.

C : compiles, crashes or corrupts silently

```
struct dma_buf *buf = dma_alloc(dev, size);
submit_dma(dev, buf);

kfree(buf);    /* freed */

/* ... 500 lines later ... */
read_completion(buf); /* UAF — undefined behaviour */
/* gcc/clang: no warning, no error */
```

Rust : compile error, zero runtime cost

```
let buf = DmaBuf::alloc(dev, size)?;
submit_dma(dev, &buf);

drop(buf);    // buf is freed here

// read_completion(&buf);
// ^^^ error[E0382]: use of moved value: `buf`
//   value used here after move
//   | drop(buf);
//   | --- value moved here
```

- The error message names the **exact line** where the value was moved and the **exact line** of the illegal use, before the code ever runs.
- no valgrind, no KASAN, no OEM execution.

2.3 Ownership prevents leaks — RAII in kernel drivers

In C, every error exit path in a driver must remember to call the right cleanup.

Rust makes this structurally impossible to get wrong.

C : common leak pattern

```
int my_driver_probe(struct pci_dev *pdev) {
    void *res_a = alloc_a();
    if (!res_a) return -ENOMEM;

    void *res_b = alloc_b();
    if (!res_b) {
        free_a(res_a); /* easy to forget */
        return -ENOMEM;
    }

    void *res_c = alloc_c();
    if (!res_c) {
        free_b(res_b); /* must remember order */
        free_a(res_a); /* and every prior alloc */
        return -ENOMEM;
    }
    /* ... */
}
```

Rust : Drop handles every exit path

```
fn my_driver_probe(pdev: &PciDev) -> Result {
    let res_a = ResourceA::alloc()?;
    // If this fails, res_a.drop() is called —
    // even on the ? early return

    let res_b = ResourceB::alloc()?;
    // res_b drops if res_c fails below

    let res_c = ResourceC::alloc()?;

    // All three are freed in reverse order
    // automatically when the function returns
    // — success or failure
    Ok(MyDriver { res_a, res_b, res_c })
}
```

- Cleanup is automatic and deterministic via scope exit, each resource implements `Drop`, ensuring release on all return paths.
- No explicit error-path cleanup logic required.
- Rust turns “developer-managed control flow problem” → “compiler-enforced lifetime and scope rule”

3. Borrowing & Lifetimes — Preventing Data Races

3.1 Borrowing : the aliasing rules formalised

Borrowing is Rust's system for temporary access without transferring ownership. This is implemented via References, which are distinct in Rust compiler view.

Shared (immutable) borrows : `&T`

- Multiple readers can coexist
- None can write while readers exist
- Maps to: read-lock held, RCU read section

```
fn print_all(items: &[DmaEntry]) { /* read-only */ }
```

```
let ring = RingBuffer::new(256);  
print_all(&ring.entries); // borrow  
print_all(&ring.entries); // another borrow — fine  
// ring is still valid and owned here
```

Data Race = aliasing + mutation + no synchronisation

Exclusive (mutable) borrow : `&mut T`

- **One** writer, **no** concurrent readers
- Maps to: write-lock held, spinlock held

```
fn add_entry(ring: &mut RingBuffer, e: DmaEntry) {  
    ring.entries.push(e); // exclusive access  
}
```

```
let mut ring = RingBuffer::new(256);  
add_entry(&mut ring, entry);  
// `ring` is fully accessible again after add_entry returns
```

The compiler **proves** that at any point in the program, either **one writer** or **N readers** holds access to any memory location, but never both. This is the data race freedom guarantee.

3.2 Lifetimes — dangling pointers eliminated

Lifetimes are the compiler's proof that a reference never outlives the data it points to. Named by the programmer, verified by the borrow checker.

Dangling pointer caught at compile time

```
fn get_ptr() -> &str {      // ← error: missing lifetime
    let local = String::from("DMA buffer");
    &local // ← would return pointer to stack data
} // `local` dropped here — pointer would dangle

// Rust requires an explicit lifetime annotation that proves
// the returned reference lives as long as the input:
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() { x } else { y }
}
// 'a = "the output lives at least as long as both inputs"
// If this invariant cannot be satisfied, the code does not compile.
```

In kernel drivers: a reference to a `struct device` inside an interrupt handler **must not outlive the device**. Lifetimes encode and verify this invariant statically. No more `dev_hold()` / `dev_put()` mismatches.

3.2.1 Rust life time in kernel :

3.3 Lifetimes prevent data races — the formal proof

Why ownership + lifetimes = data-race freedom

A data race requires **two** concurrent accesses where **at least one is a write**. The ownership model makes this impossible:

1. A **mutable** borrow `&mut T` is **exclusive** — no other borrow (mutable or shared) can exist simultaneously
2. A **shared** borrow `&T` prevents any mutable borrow from existing
3. Both are enforced statically — no runtime check, no lock

Therefore: if the code compiles, there are no data races.

RustBelt: Jung et al., POPL 2018 — formally machine-checked in Coq/Iris

RustBelt Meets Relaxed Memory: Dang et al., POPL 2020

Data race — caught at compile time

```
use std::thread;

let mut data = vec![1u8; 4096]; // DMA buffer

// Try to share mutable data across threads:
let t1 = thread::spawn(|| {
    data[0] = 0xFF; // write
});
let t2 = thread::spawn(|| {
    data[1] = 0xAA; // write concurrently
});
// error[E0502]: cannot borrow `data` as mutable
// because it is also borrowed
// error[E0373]: closure may outlive the current
// function, but it borrows `data`
```

The compiler rejects the data race **statically**. In C this compiles, runs, and silently corrupts memory — or causes a kernel panic during DMA completion.

3.4 `Send` and `Sync` — thread safety in the type system

Send: a type can be moved to another thread.

Sync: a type can be shared between threads (via `&T`).

These are **marker traits** — compile-time properties with zero runtime representation.

RustBelt §3.3: formal definition of `Send` and `Sync` via ownership predicates

```
// `Rc<T>` is NOT Send (reference counting not atomic)
// The compiler prevents it from crossing a thread boundary:
use std::rc::Rc;
let rc = Rc::new(42);
thread::spawn(move || {
    println!("{}", rc);
});
// error[E0277]: `Rc<i32>` cannot be sent between
// threads safely — use Arc<T> instead
```

Practical impact for kernel drivers

- A spinlock-protected struct is `Sync` if and only if `T: Send`
- The compiler **refuses** to put non-`Send` data in a global
- Interrupt handler shared data must be `Sync` — enforced before the module loads

Every concurrency contract that Linux documents in comments (`/* must hold lock X before accessing Y */`) Rust encodes in the **type signature** and verifies at compilation.

4. Compiler Checks — Beyond Memory Safety

4.1 What the Rust compiler verifies at every build

Memory safety (previous sections)

- No use-after-free
- No dangling references
- No data races
- No null dereferences (`Option<T>` instead of `NULL`)
- No uninitialized reads (all fields must be initialised)

Type system

- Integer overflow in debug builds → panic (not UB)
- Exhaustive `match` — all enum variants handled

```
match direction {
    DmaDir::ToDevice => { ... },
    DmaDir::FromDevice => { ... },
    // error if DmaDir::Bidirectional not handled
}
```

Error handling

- `Result<T, E>` — ignoring an error is a **compiler warning**

```
#[must_use = "this Result must be handled"]
fn map_dma(...) -> Result<SgTable, DmaError> { ... }
```

```
map_dma(dev, sg, nents); // warning: unused Result
// The kernel C equivalent: silently dropping -ENOMEM
```

- `Option<T>` — no null, no null-deref

Unsafe quarantine

- `unsafe { }` blocks are **explicitly marked**
- `grep -r "unsafe"` gives the **complete** audit surface
- Safe code cannot call `unsafe` functions accidentally

In C, all of the above require separate tools: ASAN, UBSAN, sparse, Coccinelle, clang-tidy, GCC sanitizers — none of them are mandatory, and none are exhaustive.

4.2 #[must_use] and exhaustive errors — no silent failures

A fundamental problem in C driver code: `if (ret < 0) { ... }` is optional. The compiler never complains if you forget it.

C — error silently dropped, device in bad state

```
pci_enable_device(pdev); /* returns int, ignored */
pci_set_master(pdev);   /* called even on error */

if (request_irq(irq, handler, 0, "mydrv", dev)) {
    /* forgot to check enable_device above */
    return -EIO;
}
/* device may not actually be enabled */
```

Rust — every error must be handled

```
pdev.enable()?; // ? propagates Err immediately
pdev.set_master(); // only reached if enable succeeded

pdev.request_irq(irq, handler)?;
// If any ? returns Err:
// — function returns immediately
// — Drop runs on all acquired resources
// — no resource in partially-initialised state

// Forgetting ? is a warning:
// warning: unused `Result` that must be used
```

The `?` operator is not magic — it is `match result { Ok(v) => v, Err(e) => return Err(e.into()) }`. It is purely a compile-time transformation. No exceptions, no `longjmp`, no hidden control flow.

4.3 Pattern matching — exhaustiveness by proof

`match` in Rust is **exhaustive**: the compiler proves every possible case is handled. There is no `default`: that silently swallows new enum variants.

Adding a new variant — compiler catches every callsite

```
enum PcieSpeed { Gen1, Gen2, Gen3, Gen4, Gen5 }

fn lane_bandwidth(speed: PcieSpeed) -> u64 {
    match speed {
        PcieSpeed::Gen1 => 250_000_000,
        PcieSpeed::Gen2 => 500_000_000,
        PcieSpeed::Gen3 => 985_000_000,
        PcieSpeed::Gen4 => 1_969_000_000,
        // Forgot Gen5?
        // error[E0004]: non-exhaustive patterns:
        //   `PcieSpeed::Gen5` not covered
    }
}
```

Why this matters in practice

- Add `Gen5` to the enum → **every** `match` that doesn't handle it fails to compile
- Zero chance of a silently wrong `default`: path
- Pattern matching also **destructures** and extracts data:

```
match dma_result {
    Ok(nents) if nents > 0 => {
        submit_to_hw(nents)
    },
    Ok(0)   => return Err(ErrKind::NoEntries),
    Err(e)  => return Err(e.into()),
}
```

Kernel `C switch` on an enum value: silently falls through if a new value is added. Sparse can warn about this — but only if you run it. Rust makes it a **compile error** by default.

4.4 The type system eliminates NULL

One of C's most costly design decisions: any pointer can be NULL, and dereferencing it is undefined behaviour.

```
/* C */
struct irq_data *d = irq_get_irq_data(irq);
/* d might be NULL — easy to forget the check */
d->irq_common_data.handler_data = priv;
/* null deref → BUG() → kernel panic */

// Rust: irq_get_irq_data returns Option<IrqData>
match irq_get_irq_data(irq) {
    Some(d) => d.set_handler_data(priv),
    None    => return Err(ErrKind::InvalidIrq),
}
// Compiler proved: d is valid inside Some(_)
// There is no raw pointer dereference here
```

Option<T> is the type-safe null

- None = absent value
- Some(T) = present value
- You **cannot access the inner T** without handling None first
- No accidental null dereference — it is a type error

Option<&T> is represented as a nullable pointer at the machine level — **same memory layout, zero overhead** — but the type system forces you to check it before use.

The Rust Reference: §3.1 “Niche optimisation of Option”

For kernel engineers:

container_of() macro results, device tree node lookups, optional firmware callbacks — all naturally expressed as Option<T>.

5. Modern Programming Concepts

5.1 Traits — interfaces that compose

Traits are Rust's abstraction mechanism. They are closer to Haskell type classes than to C++ virtual dispatch — and critically, they are resolved **at compile time** (zero virtual dispatch overhead).

Define the contract once

```
trait DmaDevice {  
    fn map_sg(&self, sg: &mut SgList) -> Result<usize>;  
    fn unmap_sg(&self, sg: &mut SgList);  
    fn sync_for_cpu(&self, sg: &SgList);  
}
```

Implement for any type

```
impl DmaDevice for Mlx5HcaDev {  
    fn map_sg(&self, sg: &mut SgList) -> Result<usize> {  
        // ConnectX-5 specific path  
    }  
    // ...  
}  
impl DmaDevice for VirtioNetDev { /* ... */ }
```

Static dispatch — monomorphisation

```
// Compiler generates ONE specialised version per type  
// No vtable, no indirect call, fully inlining-eligible  
fn trace_dma<D: DmaDevice>(dev: &D, sg: &mut SgList) {  
    let nents = dev.map_sg(sg)?;  
    record_event(nents, dev.name());  
}
```

Dynamic dispatch — explicit opt-in

```
// When you need runtime polymorphism:  
fn attach_tracer(dev: &dyn DmaDevice) { ... }
```

In C, this pattern requires a `struct ops { int (*map_sg)(...); ... }` with manual pointer management. Rust's trait system is the same concept — with compiler enforcement of the interface contract and zero-cost static dispatch as the default.

5.2 Zero-cost abstractions — the performance guarantee

“Zero-cost abstractions: what you don’t use, you don’t pay for. And further: what you do use, you couldn’t hand-code any better.” — Bjarne Stroustrup (C++ standard, adopted verbatim by the Rust team)

Stroustrup, “Foundations of C++”, 2012; Rust Reference: §1 “Design goals”

Iterators compile to tight loops

```
// High-level — reads like intent
let total: u64 = latencies
    .iter()
    .filter(| &&ns| ns > threshold)
    .sum();
```

// Compiles to exactly the same assembly as:

```
let mut total: u64 = 0;
for &ns in latencies {
    if ns > threshold { total += ns; }
}
```

Godbolt confirms: both emit `ADDQ` in a tight loop with no branch overhead when `threshold` is a constant.

Compiler Explorer (godbolt.org): `rustc -O2`

Generics use monomorphisation — no boxing

```
fn min_latency<T: Ord>(a: T, b: T) -> T {
    if a <= b { a } else { b }
}
// Calling with u64 generates u64-specific code
// Calling with u32 generates u32-specific code
// No virtual dispatch, no boxing, no indirection
```

Lifetime annotations are compile-time only

```
// 'a is erased before code generation
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str { ... }
// Produces the same assembly as the C equivalent
// with raw pointers — but is provably safe
```

Borrow checking and lifetime analysis are **pure static analysis** — they are stripped out before LLVM sees the code. The runtime binary is as lean as handwritten C.

5.3 Error handling — `Result<T, E>` and the `?` operator

Unlike C (errno, negative return values, `goto err_free`) or C++ (exceptions with hidden control flow), Rust makes errors **first-class values in the type signature**.

C patterns and their problems

```
int pci_probe(struct pci_dev *pdev) {
    int ret;
    ret = pci_enable_device(pdev); /* -errno or 0 */
    if (ret) goto err;
    ret = pci_request_regions(pdev, DRV_NAME);
    if (ret) goto err_disable;
    /* ... 20 more error paths ... */
err_disable:
    pci_disable_device(pdev);
err:
    return ret;
}
```

/* Problems:

- ret is untyped int; semantics undocumented
- easy to jump to wrong label
- no compiler check that all paths handled */

Rust — errors are types, `?` propagates them

```
fn pci_probe(pdev: &mut PciDev) -> Result<MyDev> {
    pdev.enable()?; // Err → return immediately
    pdev.request_regions()?; // Drop runs on previous
    let irq = pdev.alloc_irq(handler, 0)?;

    Ok(MyDev { pdev, irq })
    // On any ?, all acquired resources are
    // dropped in reverse order automatically
}

/* Benefits:
- error type is in the signature → documented
- ? is explicit, grep-able, auditable
- no goto, no labels, no cleanup lists
- compiler warns if Result is ignored */
```

Rust RFC 0243: “Trait-based exception handling” (? operator)

5.4 Fearless concurrency — async and channels

Rust's ownership model extends to asynchronous code. The same borrow checker that prevents data races in synchronous code works across `async/await` boundaries.

Channels — safe message passing

```
use std::sync::mpsc;
```

```
let (tx, rx) = mpsc::channel::<DmaEvent>();
```

```
// Producer thread — owns the tx end
thread::spawn(move || {
    tx.send(DmaEvent { latency_ns: 1842, ... }).unwrap();
});
```

```
// Consumer — owns the rx end
for event in rx {
    update_histogram(&mut hist, event.latency_ns);
}
// No shared mutable state — the type system
// prevents concurrent access to `event`
```

`Arc<Mutex<T>>` — safe shared state

```
let shared_hist = Arc::new(Mutex::new(Histogram::new()));
```

```
let hist_clone = Arc::clone(&shared_hist);
thread::spawn(move || {
    let mut h = hist_clone.lock().unwrap();
    h.record(latency_ns);
    // lock guard drops here — unlocks automatically
});
```

```
// Main thread
let h = shared_hist.lock().unwrap();
h.print_report();
```

The `Mutex<T>` **wraps** the data — you **cannot access** the histogram without holding the lock, because the data is only reachable through `lock()`. This is structurally enforced, not documented convention.

6. Performance — No Runtime Tax



What Rust does NOT have

Feature	C	Rust
Garbage collector	✗	✗
Runtime	minimal	none
Boxing by default	✗	✗
Virtual dispatch by default	✗	✗
Exception tables	✗	✗ (panic = abort)
Hidden allocations	✗	✗

Rust's borrow checking, lifetime analysis, and trait resolution are **all compile-time**. The runtime binary is indistinguishable from equivalent hand-written C. [kornel.ski/rust-c-speed — empirical analysis](https://kornel.ski/rust-c-speed)

What Rust adds over C

- **LLVM backend** — same optimiser as Clang. Identical IR, identical codegen
- **Alias information**: Rust's exclusivity rule tells LLVM that `&mut T` pointers never alias — enabling optimisations C cannot guarantee

```
// Rust can auto-vectorise this because the compiler
// PROVES src and dst do not overlap:
fn copy_dma(dst: &mut [u8], src: &[u8]) {
    dst.copy_from_slice(src);
}
// C requires `restrict` keyword and hopes the
// programmer is honest about aliasing
```

- **Panic = abort** in kernel / embedded: `#![panic_handler]` resolves to a halt — no unwind tables, no binary bloat

6.2 Benchmarks — what the numbers say

“In many benchmarks, Rust and C are within 1–3% of each other. When Rust is faster, it is usually because the aliasing information enables better auto-vectorisation. When C is faster, it is usually because a C programmer hand-optimised a specific inner loop.”

kernel.ski/rust-c-speed, 2023; [The Computer Language Benchmarks Game](#), 2024

Industrial data points

- **Cloudflare Pingora** (Rust, replaces Nginx): same throughput, **~70 % lower memory usage** per connection [Cloudflare Blog, “How we built Pingora”, 2022](#)
- **Mozilla Firefox** (Rust CSS engine, Stylo): **2× speedup** vs. C++ equivalent, fewer CVEs [Mozilla Engineering Blog, “Servo and Stylo”, 2017](#)
- **Android Binder** (Rust rewrite): **zero regressions** in performance; eliminated the UAF CVE class in IPC [Android Open Source Blog, “Memory safety for Android”, 2021](#)
- **Linux mlx5 driver tests**: Rust prototype showed no measurable overhead vs. C on throughput benchmarks [LPC 2022: “Rust in the Linux kernel” talk, kernel.org](#)

The aliasing advantage — concrete example

```
// C: compiler cannot vectorise safely
// — it cannot prove buf_in ≠ buf_out
void add_headers(u8 *buf_out, const u8 *buf_in,
                size_t n) {
    for (size_t i = 0; i < n; i++)
        buf_out[i] = buf_in[i] | 0x80;
}

// Must use `restrict` to get auto-vectorisation

// Rust: exclusive mutable borrow PROVES no overlap
// Compiler auto-vectorises without any annotation
fn add_headers(out: &mut [u8], in_buf: &[u8]) {
    for (o, &b) in out.iter_mut().zip(in_buf) {
        *o = b | 0x80;
    }
}
```

The **same type-system property** that prevents data races **also enables better codegen**. Safety

6.2 Benchmarks — what the numbers say

and performance are not in tension — they arise from the same source.

7. The Ecosystem — Rust is Reshaping the Toolchain

7.1 A language that attracted a new toolchain generation

Rust's safety guarantees and modern ergonomics have attracted developers to build **entire new categories of tooling** in it — not just systems software. These tools now affect your daily workflow whether you use Rust or not.

These slides are written in Rust

- **Typst** — next-generation document system (these slides)
Zero-config, sub-second compile, Git-friendly plain text
Written entirely in Rust (typst.app)

Your build pipeline

- **ripgrep** (rg) — faster grep (github.com/BurntSushi/ripgrep)
- **fd** — faster find
- **bat** — syntax-highlighting cat
- **Ruff** — Python linter, 10–100× faster than pylint (astral.sh/ruff)
- **uv** — Python package manager, replaces pip
- **swc** — JS/TS compiler, 70× faster than Babel

Critical infrastructure, written in Rust

- **Cloudflare Pingora** — processes 1 trillion requests/day ([Cloudflare Blog, 2022](#))
- **AWS Firecracker** — microVM hypervisor powering Lambda & Fargate ([NSDI 2020](#))
- **Google crosvm** — ChromeOS VM monitor
- **Microsoft Azure** — hypervisor components
- **1Password** — password manager core

The meta-point for this audience

Rust adoption signals a shift in how systems software is written. Teams that invest now — especially in kernel driver work where safety guarantees directly translate to CVE reduction — will have a measurable engineering advantage within 2–3 years.

Formal verification of Rust's type system

- Jung, R., Jourdan, J-H., Krebbers, R., Dreyer, D. **"RustBelt: Securing the Foundations of the Rust Programming Language."** *Proc. ACM Program. Lang.* 2, POPL, Article 66. January 2018. plv.mpi-sws.org/rustbelt/pop18
- Dang, H-H., Jourdan, J-H., Kaiser, J-O., Dreyer, D. **"RustBelt Meets Relaxed Memory."** *Proc. ACM Program. Lang.* 4, POPL, Article 34. January 2020.
- Grannan, Z., Bílý, A., et al. **"Place Capability Graphs: A General-Purpose Model of Rust's Ownership and Borrowing Guarantees."** arXiv:2503.21691, 2025.
- Silva, T., Bispo, J., Carvalho, T. **"Foundations for a Rust-Like Borrow Checker for C."** LCTES 2024. ACM.
- Li, J., et al. **"Formally Understanding Rust's Ownership and Borrowing System at the Memory Level."** *Formal Methods in System Design*, Vol. 64, Issue 1. Springer, July 2024.

Industry data — memory safety statistics

- Microsoft Security Response Centre. **"A Proactive Approach to More Secure Code."** Gavin Thomas, 2019.
- Gaynor, A., Thomas, G. **"Memory Safety in the Linux Kernel."** Linux Security Summit, 2019.
- Google Project Zero. **"A Survey of Memory Safety Issues in Chrome."** 2020. googleprojectzero.blogspot.com
- Android team. **"Memory Safety in Android."** Android Open Source Blog, 2021.

Performance analysis

- Paczos, K. **“The Speed of Rust vs C.”** kornel.ski/rust-c-speed, 2023.
- The Computer Language Benchmarks Game. benchmarksgame-team.pages.debian.net, 2024.
- Compiler Explorer (Godbolt). Rustc -O2 output verification. godbolt.org

Production deployments

- Cloudflare. **“How We Built Pingora, Our Rust-based HTTP Proxy.”** Cloudflare Blog, 2022. blog.cloudflare.com
- Agache, A., et al. **“Firecracker: Lightweight Virtualization for Serverless Applications.”** NSDI 2020. USENIX.
- Mozilla. **“Shipping Rust in Firefox: The Quantum CSS / Stylo Story.”** Mozilla Engineering Blog, 2017.

Standards and language specifications

- The Rust Reference. doc.rust-lang.org/reference, 2024.
- Rust RFC 0243. **“Trait-based exception handling.”** rust-lang.org/RFCs/0243.
- Stroustrup, B. **“Foundations of C++.”** ESOP 2012. Springer.

Kernel-specific

- Ojeda, M., et al. **“Rust in the Linux Kernel.”** LPC 2022 talk. kernel.org
- Linux Kernel Documentation: [Documentation/rust/](https://documentation/rust/). docs.kernel.org/rust
- Corbet, J. **“Rust Goes Mainstream in the Linux Kernel.”** LWN.net, December 2025.

The borrow checker is not a restriction.

It is a proof system that tells you, at submission time, that your driver does not have a use-after-free, a data race, a missing error path, or an uninitialised pointer.

Every kernel bug it catches is a CVE that never ships.

Thank you.

Questions?